

Tutorial: Using Sessions in Node.js with Express

1. Introduction to Sessions

1.1 What Are Sessions?

Sessions store **temporary data on the server**, unlike databases which store data permanently (e.g., MySQL, MongoDB). Session data exists until:

- A specified timeout expires, or
- The server manually deletes the session.

1.2 Why Use Sessions?

Once stored, session data can be accessed across any page of the website as long as it exists.

1.3 Real-World Use Cases

1. User Authentication

- After login, a session is created and used to show username, logout button, etc.
- On logout, the session is destroyed.

2. Shopping Carts

- Cart items are saved temporarily in the session until checkout.

3. Flash Messages

- Display success/error messages between redirects.

4. Multi-step Forms

- Save step-by-step data across multiple pages until final submission.

5. User Preferences

- Language or currency selection stored in sessions for consistent display.

6. Form Data Preservation

- When validation fails, re-fill form fields using saved session data.

7. Captcha Validation

- Captcha codes stored in session to verify user correctness across pages.

2. Installing and Configuring express-session

2.1 Installation

```
npm install express-session
```

2.2 Basic Setup in Express

```
import express from "express";
const app = express();

import session from "express-session";

app.use(
  session({
    secret: "secret password",
    resave: false,
    saveUninitialized: false,
    cookie: {
      maxAge: 1000 * 60 * 60 * 24 // 24 hours
    }
  })
);
```

2.3 Meaning of Configuration Fields

Key	Description
<code>secret</code>	Encryption key stored in cookies to protect from session hijacking.
<code>resave</code>	Whether to forcibly save session back to store. Usually <code>false</code> .
<code>saveUninitialized</code>	Session is not created unless data is added.
<code>cookie.maxAge</code>	Timeout duration in milliseconds. Example: 24 hours.

3. Working with Sessions

3.1 Creating/Storing a Session

```
app.get("/set-username", (req, res) => {
  req.session.username = "Yahoo Baba";
  res.send("Username has been set in session");
});
```

3.2 Reading a Session

```
app.get("/get-username", (req, res) => {
  if (req.session.username) {
    res.send(`Username from session is: ${req.session.username}`);
  } else {
    res.send("No username found in session");
  }
});
```

3.3 Access Session Anywhere

You can read session data on:

- Home route
- About page
- Any other page or API route

3.4 Destroying a Session (e.g., Logout)

```
app.get("/destroy", (req, res) => {
  req.session.destroy((err) => {
    if (err) {
      return res.status(500).send("Failed to destroy session");
    }
    res.send("Session destroyed successfully");
  });
});
```

4. Storing Sessions in MongoDB (Instead of RAM)

4.1 Why Store Sessions in MongoDB?

- In-memory sessions fill RAM quickly when many users visit.
- MongoDB provides persistent, scalable storage for session data.

4.2 Installing Required Package

```
npm install connect-mongo
```

4.3 Setting Up MongoDB Session Store

```
import session from "express-session";
import MongoStore from "connect-mongo";
```

```
app.use(
  session({
    secret: "secret password",
    resave: false,
    saveUninitialized: false,
    store: MongoStore.create({
      mongoUrl: "mongodb://127.0.0.1:27017/sessionDB",
      collectionName: "mySessions"
    }),
    cookie: {
      maxAge: 1000 * 60 * 60 * 24
    }
  })
);
```

Notes:

- `sessionDB` is auto-created if not present.
- `mySessions` collection is auto-generated.
- You can customize TTL inside MongoStore as well.

Example:

```
store: MongoStore.create({
  mongoUrl: "...",
  collectionName: "mySessions",
  ttl: 1000 * 60 * 60 * 24
})
```

5. Verifying Sessions in MongoDB

5.1 Check Databases

```
show dbs
```

5.2 Switch Database

```
use sessionDB
```

5.3 View Session Collection

```
show collections
```

5.4 View Session Documents

```
db.mySessions.find()
```

Document contains:

- `_id`
- `expires`
- Session data (e.g., `{ "username": "Yahoo Baba" }`)

6. Summary of Key Concepts

- Sessions store temporary server-side data.
 - Use `express-session` to create, read, and delete sessions.
 - Sessions can hold any key-value data.
 - Use `connect-mongo` to store sessions inside MongoDB.
 - MongoDB session storage prevents RAM overload and improves scalability.
-